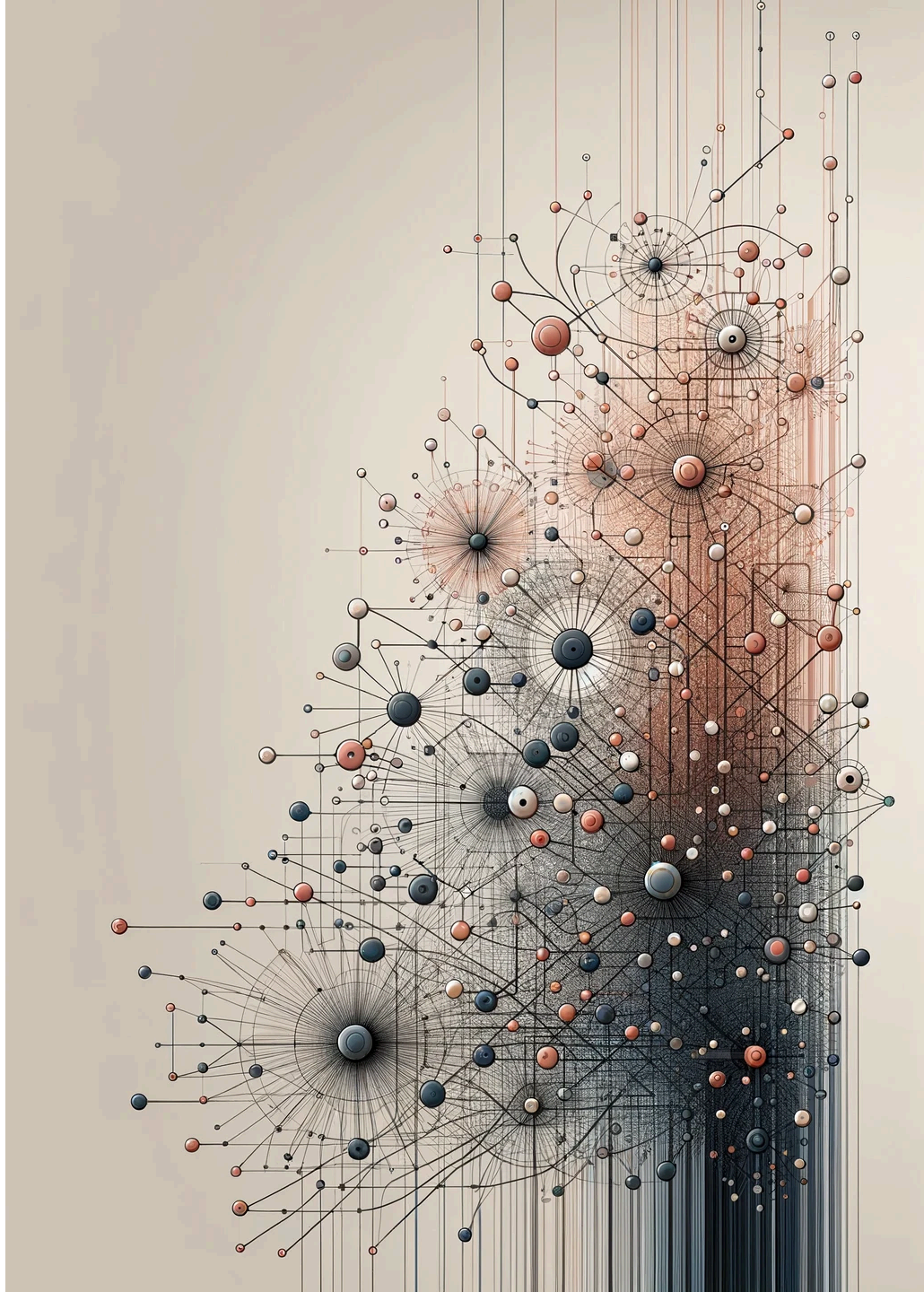
UNIVERSITÀ DI TORINO

Analisi e Visualizzazione delle Reti Complesse

NS21 - Representation Learning on Graphs

Prof. Rossano Schifanella



UNIVERSITÀ DI TORINO

Outline

1. **From hand-crafted features to learned representations** — the motivation
2. **How embeddings are learned** — the encoder–decoder framework
3. **How to measure similarity** — proximities and scalability
4. **Random walks and Skip-gram** — the training engine
5. **DeepWalk** — the first complete algorithm
6. **node2vec** — biased walks, two interpretable knobs
7. **From nodes to whole graphs** — pooling, supernodes
8. **Evaluating embeddings** — link prediction, node classification, visualisation
9. **Limitations and the road to Graph Neural Networks**

UNIVERSITÀ DI TORINO

Section 1 — From hand-crafted features to learned representations

In this section:

- How did people apply ML to networks before embeddings?
- Why does that approach fall short?
- What does it mean to **learn** a representation, and what is an **embedding**?

By the end: a clear motivation for learning vectors directly from the graph.

Traditional feature engineering

For decades, applying machine learning to networks meant **designing features by hand**:

- Pick a list of structural descriptors: degree, clustering coefficient, betweenness, PageRank, k-core number, motif counts, ...
- Compute them for every node (or pair of nodes, for link tasks)
- Feed the resulting vectors into classical models — logistic regression, SVMs, random forests, gradient boosting

Feature engineering — a worked example

Predicting influential users in a social network:

1. Compute degree, PageRank, community membership, eigenvector centrality for every account
2. Stack them into a feature matrix $X \in \mathbb{R}^{|V| \times k}$
3. Train an SVM on a labelled subset of "influential / not influential" users
4. Predict the rest

This works — but every step in the pipeline depends on **human intuition** about which descriptors matter.

Why hand-crafted features fall short

- **Labour-intensive**: a new task means a new round of feature engineering
- **Domain expertise required**: knowing what to compute presupposes knowing the network well
- **Poor transfer**: features designed to predict influence rarely also help predict community membership or future links
- **Ceiling on expressiveness**: simple statistics cannot capture intricate, multi-hop dependencies that often drive behaviour
- **Brittle to network type**: descriptors that work for social graphs may be useless for molecules or roads

A case where hand-crafted features fail

In protein–protein interaction networks, two proteins can have very different degrees yet share the same biological function because they sit in equivalent **structural positions**.

No single hand-crafted statistic captures this — but a **learned embedding** can.

A new paradigm — *learn* the features

Representation learning flips the pipeline:

- We **do not pre-specify** which descriptors matter
- We let an algorithm **discover** a vector representation $\mathbf{z}_v \in \mathbb{R}^d$ for each node directly from the graph
- The same representation can then be used for **many downstream tasks**: classification, link prediction, clustering, recommendation, anomaly detection

Why this paradigm shift matters

Two consequences worth emphasising:

1. **Features emerge from the data**, not from human guesswork
2. **A single embedding, many uses** — the same vectors can power a recommender system today and a fraud detector tomorrow

The rest of this lecture builds up the machinery that makes this possible.

What is an embedding?

An **embedding** is a mapping from a discrete object (a word, an image, a node, a graph) to a vector of real numbers, with the property that **objects judged "similar" by some criterion end up close in vector space**.

Once we have such a mapping, the embedding vectors become a drop-in input to any machine learning algorithm — k-NN, logistic regression, neural networks — none of which know anything about the original graph.

The next slide lists the **three flavours** of graph embedding, each useful for different tasks.

Three flavours of graph embedding

Graph embeddings come in three families, distinguished by *what* is embedded:

- **Node embeddings** — one vector per node *(the focus of this lecture)*
  Applications: node classification, link prediction, recommendation, clustering
- **Edge / relation embeddings** — one vector per edge or per relation type
  Applications: knowledge graphs (Freebase, WordNet), multi-relational networks
- **Graph embeddings** — one vector for an entire graph
  Applications: molecule property prediction, graph classification, graph similarity search

Section 2 — How embeddings are learned

In this section:

- What is the minimum machine learning we need?
- How do we go from a long binary list (a row of A) to a short, useful vector?
- What is the **encoder–decoder framework** that unifies every method in this lecture?

By the end: a 4-ingredient recipe that every algorithm we'll see is just a different choice of.

Machine learning, in one slide

The whole lecture rests on three simple ideas. We will use them again and again.

Vectors and dot product. Each node will get a vector $\mathbf{z}_v$ — just a list of d numbers, e.g. $(0.3, -0.7, 0.1)$. The **dot product** $\mathbf{z}_u^\top \mathbf{z}_v$ multiplies the two lists coordinate-by-coordinate and sums the result. It is **large and positive** when the vectors point in the same direction (similar) and **negative** when they point in opposite directions (dissimilar). This is our measure of similarity in vector space.

Machine learning — loss and gradient descent

Loss. A single number that says how *wrong* the vectors currently are. We define it so that "good vectors" → small loss and "bad vectors" → large loss.

Gradient descent — the training loop. Repeat many times: compute the loss, then nudge each vector slightly in the direction that makes the loss smaller. After enough small nudges, the vectors stop moving. Done.

Bottom line. Every method in this lecture is a different choice of *what "similar" means in the graph* and *how to make the nudges fast*.

From a long binary list to a short vector

The "raw" representation of a node is the row of the adjacency matrix — a binary list saying *who am I connected to?*

- A graph with 1 million nodes gives a 1-million-long list per node, almost entirely zeros
- Too long, too sparse, and too redundant to feed into a machine learning model

An embedding compresses that list down to a short vector — typically just **64 to 256 numbers** — while keeping what matters about the graph.

Raw vs. embedding — at a glance

	"Raw" representation	Embedding
Length	one number per node in the graph (millions)	small and fixed (e.g. 128)
Content	mostly zeros	dense, real-valued
Comparison	rows do not reflect graph distance	similar nodes → vectors close by
Use	impractical	feeds directly into any ML algorithm

Standard dimensionality reduction (PCA on A) is not enough: graphs encode multi-hop and community structure that simple compression discards.

What does "similar nodes" mean? — Proximity orders

Different embedding methods make different choices about which kind of similarity to preserve.

The standard taxonomy:

- **First-order proximity** — direct edges
  Connected nodes should have similar vectors.
  Example: friends in a social network.
  Captures: the immediate neighbourhood — who you know directly.

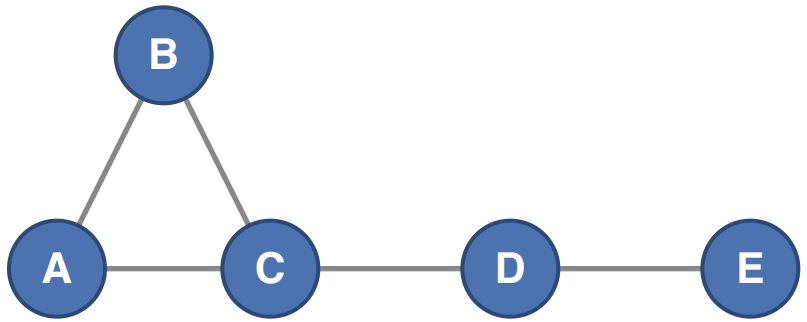
Proximity orders — second & higher

- **Second-order proximity** — shared neighbourhoods
  Nodes with many common neighbours should be similar, even if not directly connected.
  Example: two professors who collaborate with the same researchers but never with each other.
  Captures: roles, contexts, "friends-of-friends" patterns.
- **Higher-order / global structure** — multi-hop paths
  Preserve overall network distances, communities, hierarchical organisation.
  Captures: position within the network as a whole — bridges, peripheries, hubs.

The choice of proximity is the **most consequential design decision** in any embedding method — it determines which downstream tasks the embeddings will be good at.

Worked example — proximities (1)

Five nodes, edges $(A, B), (A, C), (B, C), (C, D), (D, E)$. $\{A, B, C\}$ forms a triangle around hub C; D bridges to a leaf E.



First-order — direct connection (A_{uv}):

- (A, B): connected → 1
- (A, D): not connected → 0
- (A, E): not connected → 0

Worked example — second-order proximity

Common neighbours ($(A^2)_{uv}$, count of length-2 paths):

- (A, D): path $A \to C \to D$ → 1 — they share neighbour C, even with no direct edge
- (B, D): path $B \to C \to D$ → 1
- (A, E): no length-2 path → 0

Worked example — proximities (2)

Same 5-node graph: edges $(A, B), (A, C), (B, C), (C, D), (D, E)$.

Third-order — three-step paths ($(A^3)_{uv}$):

- (A, E): path $A \to C \to D \to E$ → 1 — finally captured
- (B, E): path $B \to C \to D \to E$ → 1

The point. The same pair (A, E) has similarity 0, 0, and 1 under the three orders. **Choosing which proximity to preserve in the embedding decides which pairs end up close in vector space.**

The encoder–decoder framework

Almost every node-embedding method follows the **same four-step recipe**:

1. **Encoder** — a rule that turns each node into a vector $\mathbf{z}_v$
   Simplest rule: a lookup table that says "node 7 → these d numbers".

2. **Graph similarity** — a target value $s(u, v)$ saying how related two nodes are *in the graph itself*
   Examples: 1 if they share an edge, the number of paths between them, the chance that a random walk from u visits v.

Encoder–decoder framework — decoder & loss

3. Decoder — a way to reconstruct that target value *from the two vectors*

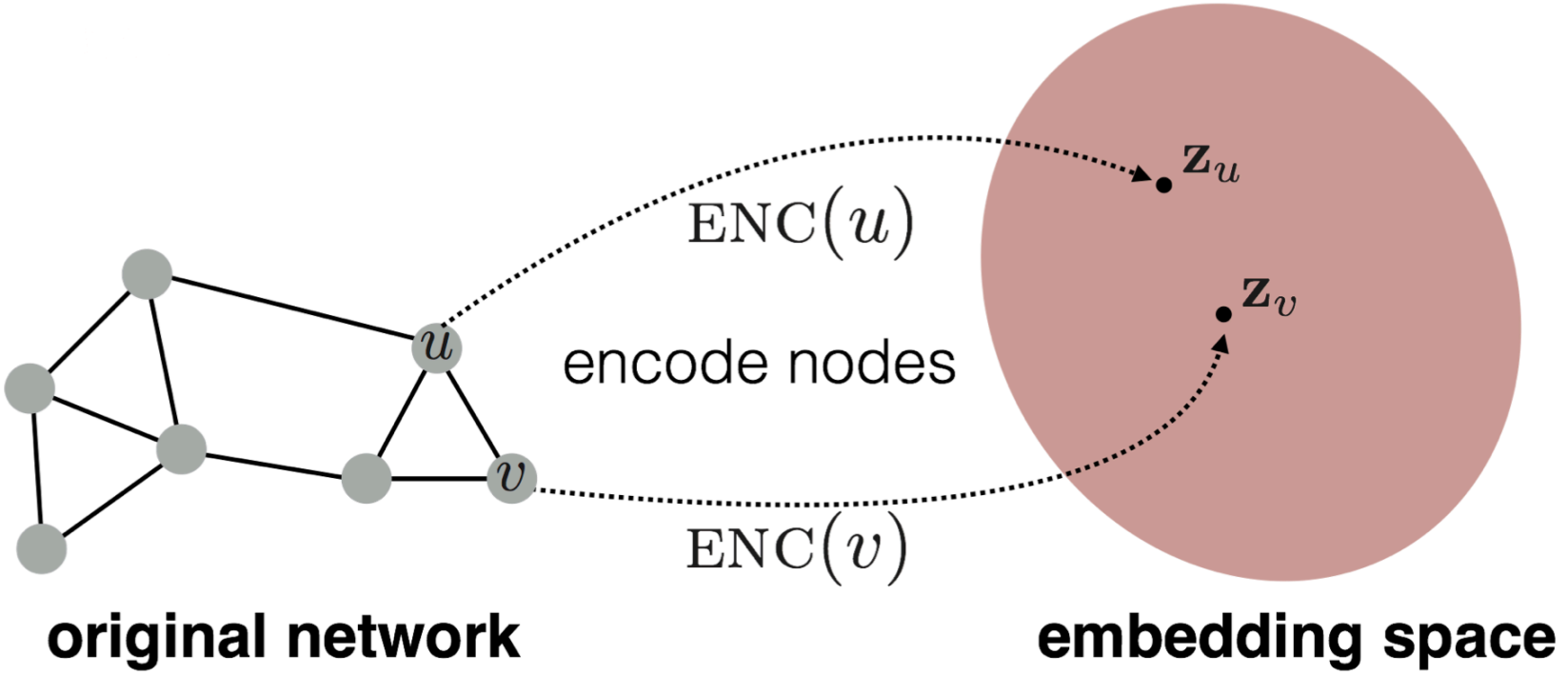
Default choice: the **dot product** $\mathbf{z}_u^\top \mathbf{z}_v$.

4. Loss — one number measuring how wrong the vectors currently are

Default choice: the squared mismatch $\left(\mathbf{z}_u^\top \mathbf{z}_v - s(u,v)\right)^2$, summed over node pairs.

> **Read the framework like this.**
> The target $s(u,v)$ is the **similarity in the graph** (the truth).
> The decoder $\mathbf{z}_u^\top \mathbf{z}_v$ is the **similarity in vector space** (the prediction).
> Training picks the vectors that make the gap between the two as small as possible.

Every method below — DeepWalk, node2vec — is a different choice of these four ingredients.


$\mathbf{z}_u$
$\mathbf{z}_v$
$\mathrm{ENC}(u)$
encode nodes
$\mathrm{ENC}(v)$
u
v
original network
embedding space

Worked example — setup

A concrete instance of the framework on a **4-node square graph** with edges $(1,2), (2,3), (3,4), (4,1)$.

- **Encoder.** Each node gets a random vector of $d = 2$ numbers:

$$\mathbf{z}_1 = (0.1,\ 0.2), \quad \mathbf{z}_2 = (-0.3,\ 0.4), \quad \mathbf{z}_3 = (0.5,\ -0.1), \quad \mathbf{z}_4 = (0.0,\ 0.3)$$

- **Target** $s(u, v)$ — the value we *want* the decoder to produce:
  - $s(u, v) = 1$ if u and v share an edge
  - $s(u, v) = 0$ otherwise
- **Decoder** — predicts the target from the two vectors: $\mathbf{z}_u^\top \mathbf{z}_v$
- **Loss for one pair** — how far the prediction is from the target: $\left(\mathbf{z}_u^\top \mathbf{z}_v - s(u, v)\right)^2$

The next slide runs **one update step** on this setup.

Worked example — one update step

Pick the pair $(1, 2)$. They share an edge → **target** $s(1, 2) = 1$.

1. **Decoder output** (dot product): $\mathbf{z}_1^\top \mathbf{z}_2 = (0.1)(-0.3) + (0.2)(0.4) = 0.05$
2. **Mismatch** (decoder – target): $0.05 - 1 = -0.95$ — way below where we want it
3. **Nudge** $\mathbf{z}_1$ a small step toward $\mathbf{z}_2$ (and vice-versa); after the nudge $\mathbf{z}_1 \approx (0.07,\ 0.24)$
4. The new dot product moves up — slightly closer to the target 1.

Repeat for every pair in the graph:

- **edge pairs** (target 1) → push the two vectors **together**
- **non-edge pairs** (target 0) → push the two vectors **apart**

After many passes, connected nodes have aligned vectors; non-connected ones don't.

This loop sits at the core of every method in the lecture.

Checkpoint — you now have the scaffold

You have just seen the **complete skeleton** of every node-embedding method:

1. Pick a similarity target $s(u, v)$
2. Encode each node as a vector $\mathbf{z}_v$
3. Decode (dot product) to predict the target
4. Define a loss; nudge the vectors with gradient descent

Everything that follows in this lecture is **filling in choices** for these four boxes.

> **One thing to remember:** every method = encoder + similarity + decoder + loss.

Two families of encoders — shallow vs. deep

Shallow encoders *(the focus of this lecture)*

- Just a **lookup table** — node ID in, vector out
- Each node has its own independent vector
- Number of parameters grows with the **number of nodes** in the graph
- Examples: DeepWalk, node2vec

Two families of encoders — deep encoders

- A neural network computes the embedding from a node's features and its neighbours
- Parameters are **shared** across nodes — the model has the **same size** whether the graph has 1,000 or 1 million nodes
- Examples: Graph Convolutional Networks (GCN), GraphSAGE, Graph Attention Networks (GAT), GIN

Shallow vs. deep encoders — comparative table

Aspect	Shallow Encoders	Deep Encoders
Training	Faster, simpler	More complex, requires tuning
Model size	grows with number of nodes	independent of number of nodes
Node features	Ignored	Fully leveraged
New nodes?	retrain from scratch	run the same network on the new node
Bottleneck	memory (one vector per node)	computation (message passing)
Best for	Static graphs, simple downstream tasks	Dynamic graphs, feature-rich domains

Section 3 — How to measure similarity

In this section:

- What concrete choices of "similar nodes in the graph" can we use as a target?
- Why can't we just use *all pairs* of nodes — what blows up?

By the end: a menu of similarity targets and the realisation that we need a smarter way to train than "compare every pair".

Picking the similarity — a menu

Choice	What it asks	What it captures
Direct edge	Are u and v neighbours?	immediate connection
Shared neighbours	How many friends do u and v have in common?	friend-of-friend
Length-k paths	How many paths of length k between u and v?	multi-hop reach
Katz index	All paths, longer ones counted less	mixed local + global
Personalised PageRank	Probability that a random walk starting at u visits v	random-walk influence

Different answers, same question

All of these answer the same question — *how related are u and v?* — but they answer differently, and each answer produces a different embedding.

The choice of similarity is what makes one embedding good for community detection and another good for link prediction.

The scalability problem

The "naive" plan — sum the per-pair loss over **all pairs of nodes** — has two costs that explode on real graphs:

1. **Too many pairs.** N nodes → N^2 pairs. For a million-node graph, that is **a trillion pairs** per training pass.
2. **Expensive similarities.** Some choices of similarity (Katz, personalised PageRank) require their own heavy computations for every pair.

For graphs with millions of nodes, computing the loss over all pairs is **infeasible**. We need an approximation that touches only a small fraction of the pairs at each update.

Section 4 — Random walks and Skip-gram

In this section:

- How do we generate "good pairs" cheaply? → **random walks**
- How do we measure success on those pairs? → **Skip-gram** (a trick borrowed from language models)
- How do we make training fast on huge graphs? → **negative sampling**

By the end: the full machinery behind DeepWalk and node2vec.

Random walks — definition and intuition

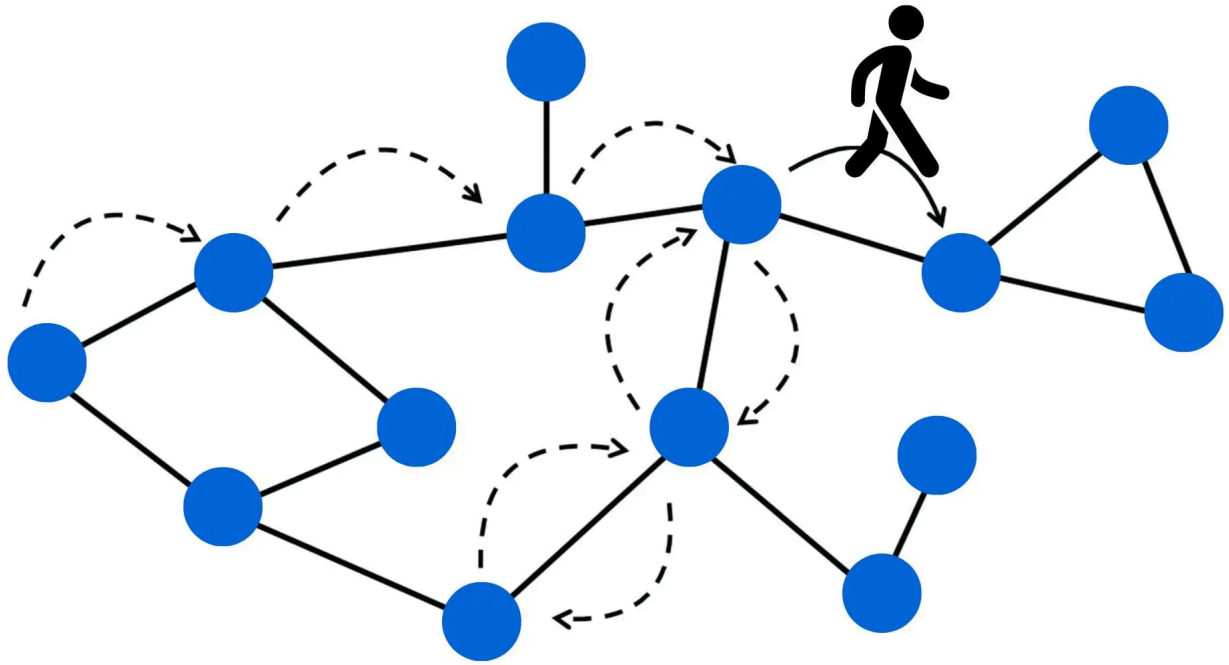
A **random walk** on a graph G is a stochastic process:

- Start at a node u_0
- At step t, jump to a neighbour of the current node u_t, chosen according to some transition rule
- Record the visited sequence $u_0, u_1, u_2, \ldots, u_L$

The walk's length L controls how far it explores; the transition rule controls *what* it explores (uniform neighbour, biased, weighted, ...).

Why walks define a good similarity

1. **Flexible context** — nodes that co-occur in walks are operationally "related"; the definition adapts to the graph
2. **Local + global balance** — short walks stay in communities, long walks reach distant structural roles
3. **Cheap to compute** — generating one walk costs $O(L)$, no matrix inverses, no eigendecompositions
4. **Implicit higher-order information** — co-occurrence at distance k in a walk encodes a k-step relationship without computing A^k



From walks to a learning target

Run many random walks. For each node u, collect every other node v that shows up nearby in the walks — call this the **walk-neighbourhood** of u, written $N(u)$.

The embedding problem now has a concrete target:

> **Choose vectors so that, for every node u, the nodes in its walk-neighbourhood $N(u)$ are easy to predict from $\mathbf{z}_u$.**

Concretely:

- If v appears often near u in the walks → push $\mathbf{z}_u^\top \mathbf{z}_v$ up
- If v never appears near u in the walks → keep $\mathbf{z}_u^\top \mathbf{z}_v$ small

That is the entire learning signal. The next slide draws the analogy to language, where the very same idea was invented for words.

The word–node analogy and the Skip-gram objective

Word embeddings (Word2Vec, Mikolov et al. 2013) learn vectors for words by predicting which words appear together in sentences. The same idea, transplanted to graphs, becomes node embeddings.

Language ↔️ networks — parallel pipelines

Language	Networks
Sentence — sequence of words	Random walk — sequence of nodes
Context window around a target word	Window around a target node in the walk
"Bank" near "money" / "loan" / "river"	A person near friends / colleagues / family
Distributional hypothesis: meaning from context	Structural hypothesis: role from walk-context

Nodes that appear in similar walk-contexts develop similar embeddings — and this naturally captures community structure and functional similarity.

Skip-gram — turning the target into a probability

How do we turn *"the dot product should be large for context pairs"* into something we can train? We convert it into a **probability** that node v is the next-seen context of node u.

The standard recipe is the **softmax**:

$$P(v \mid u) \;=\; \frac{\exp(\mathbf{z}_u^\top \mathbf{z}_v)}{\text{sum of } \exp(\mathbf{z}_u^\top \mathbf{z}_n) \text{ over all nodes } n}.$$

Reading this in plain language:

- **Numerator** — how aligned $\mathbf{z}_u$ is with $\mathbf{z}_v$ (large when they point the same way)
- **Denominator** — the same alignment summed against **every other node** in the graph (a normalisation, so the fraction is between 0 and 1)
- The fraction is now a probability — we can train by making it large

Skip-gram — training rule and bottleneck

Training rule. For every (target, context) pair (u, v) from the walks, push $P(v \mid u)$ towards 1.

The bottleneck. That denominator forces us to compare u against **every node**, for every pair we train on. On a million-node graph: 10^{12} such comparisons per pass — infeasible. The next slide is how we get rid of it.

> **Remember:** *make true context pairs probable; the obstacle is the denominator.*

Negative sampling — the rule

For each (target, context) pair (u, v) from the walks, plus K randomly picked "negative" nodes $n_1, \ldots, n_K$:

- **Reward** the true pair → push $\mathbf{z}_u^\top \mathbf{z}_v$ **up**
- **Penalise** each random pair → push $\mathbf{z}_u^\top \mathbf{z}_{n_k}$ **down**

Negative sampling — squashing the dot product

A small detail — squashing. Dot products can be any real number, but we want to read them as "similar / not similar". The **logistic function** $\sigma(x) = 1/(1 + e^{-x})$ does the job:

- $\sigma(\text{large positive}) \approx 1$ — confidently "yes, similar"
- $\sigma(0) = 0.5$ — undecided
- $\sigma(\text{large negative}) \approx 0$ — confidently "not similar"

Cost per update: $1 + K$ comparisons (with $K \approx 10$) instead of comparing against every node — **that is the speed-up**.

> **Remember:** *replace the expensive denominator with a few random comparisons.*

Negative sampling — a worked numerical pair
Suppose, mid-training:

- **True pair** (target u, context v): $\mathbf{z}_u^\top \mathbf{z}_v = 1.5$ → $\sigma(1.5) \approx 0.82$
  Already pretty aligned — but the loss still says *"make it more so"*.
- **Random negative** n: $\mathbf{z}_u^\top \mathbf{z}_n = 0.3$ → $\sigma(0.3) \approx 0.57$
  Too aligned for a random pair — the loss says *"push apart"*.

One update step nudges $\mathbf{z}_u$:

- closer to $\mathbf{z}_v$ (reward the true pair)
- away from $\mathbf{z}_n$ (penalise the random one)

After many such updates over the walk corpus, embeddings stabilise: nodes that frequently co-occur land close together; random pairs stay far apart.

Checkpoint — you now have a fast training loop

We have built a complete training engine: **walks** generate cheap pairs, **Skip-gram** turns them into a learning target, **negative sampling** keeps each update fast.

The only remaining question is: **what walks do we actually run?** The next two sections are two answers — DeepWalk (uniform) and node2vec (biased with two knobs).

> **One thing to remember:** walks + Skip-gram + negative sampling is the **engine**; choosing the walk strategy is the **knob**.

Section 5 — DeepWalk

The first complete algorithm built on the machinery of Section 4.

- **Origin** — Perozzi, Al-Rfou, Skiena (KDD 2014); the first major graph-embedding method
- **Core idea** — nodes appearing in similar contexts in random walks should have similar embeddings
- **Inspiration** — directly transplants Word2Vec's Skip-gram from text to graphs
- **Analogy** — a random walk is a "sentence", a node is a "word"
- **Reference** — DeepWalk: Online Learning of Social Representations (KDD 2014)

DeepWalk — Step 1: generate walks

For each node v in the graph:

- Start a walk from v
- For L steps, move to a **uniformly random neighbour** — transition probability $1/\deg(\text{current node})$
- Repeat r times per node

The original paper used $r = 80$ walks per node (Perozzi et al., §6.1); modern reimplementations and node2vec settle on $r = 10,\ L = 80$.

Result. A *corpus* of $r \cdot |V|$ node sequences — the graph analogue of a text corpus, where each walk is a "sentence" and each node is a "word".

DeepWalk — Step 2: build context pairs

For each walk, and for each position i along it:

- Slide a window of half-width w (typically $w = 5$ or 10) over the walk
- Every node v_j inside the window (with $j \neq i$) becomes a **context** of the target v_i
- Treat the resulting pair (v_i, v_j) as one training example

Each walk of length L yields roughly $2wL$ pairs.

The corpus is now a long list of (target, context) pairs, ready for Skip-gram training on the next slide.

DeepWalk — Steps 3 & 4: train and read out

Step 3 — Apply Skip-gram.

- Initialise the embedding matrix Z randomly
- For each (target, context) pair, nudge their embeddings *closer*; for a few random non-context nodes, nudge them *apart*
- Repeat over many passes until embeddings stabilise

Step 4 — Output. Each row of the trained Z is a d-dimensional embedding, ready for any downstream classifier or similarity computation.

Historical note. The original DeepWalk paper (2014) used **hierarchical softmax** to dodge the bottleneck; node2vec (2016) switched to negative sampling, which is now standard for both.

DeepWalk — end-to-end on a toy graph

Graph. Two 4-node cliques $C_1 = \{1, 2, 3, 4\}$ and $C_2 = \{5, 6, 7, 8\}$, joined by a single bridge edge $(4, 5)$.

Step 1 — generate walks. With $r = 2$ walks per node and $L = 6$, two example walks rooted at node 1:

- $1 \to 3 \to 2 \to 4 \to 5 \to 7$ — crosses the bridge
- $1 \to 2 \to 4 \to 3 \to 1 \to 2$ — stays in C_1

Both kinds happen, in proportion to how the bridge connects.

DeepWalk on a toy graph — Steps 2 & 3

Step 2 — context pairs with window $w = 2$. From the first walk, for target node 4 at position 3:

$$(4, 3), \quad (4, 2), \quad (4, 5), \quad (4, 7).$$

Targets near community boundaries pick up cross-community context — this is exactly what mixes them in embedding space.

Step 3 — Skip-gram with negative sampling. For each pair, run the gradient step from the previous slide; for K random negatives, push apart. Repeat over all walks for several epochs.

DeepWalk on a toy graph — Step 4: output

A trained $Z \in \mathbb{R}^{8\times 2}$. Plotting the rows: nodes 1–4 form one cluster, nodes 5–8 form another, with nodes 4 and 5 slightly closer to the boundary.

Community structure has emerged from local walk co-occurrences alone — the algorithm was never told there were two communities.

Checkpoint — your first complete algorithm

You have just walked through **DeepWalk** end-to-end on a real graph — a **4-line procedure** built on top of three ideas you already know (walks, Skip-gram, negative sampling).

> **One thing to remember:** DeepWalk = uniform random walks + Skip-gram + negative sampling.

DeepWalk — hyperparameters and what they control

Hyperparameter	Default	What it controls
d — embedding dimension	64–256	Capacity vs. overfitting; downstream cost
L — walk length	40–80	How far context can reach
r — walks per node	10–20	Statistical coverage of the corpus
w — context window	5–10	What counts as "near" in a walk
k — negative samples	5–20	Cost of each gradient step (if SGNS used)

Rule of thumb. Longer walks and larger windows push the embeddings towards capturing **global / community** structure; shorter walks and smaller windows emphasise **local / first-order** structure.

DeepWalk — strengths

- **Conceptual simplicity** — one short paper, one short algorithm
- **Scales** to graphs with millions of nodes thanks to walks + negative sampling
- **No feature engineering** — only the graph structure is needed
- **Versatile** — performs reasonably on social, citation, biological, and web graphs
- **Drop-in vectors** — outputs feed directly into any sklearn-style classifier

DeepWalk — what's missing?

Two limitations are specific to DeepWalk's walk strategy and **directly motivated node2vec**:

- **Uniform exploration** — every neighbour is equally likely; no way to favour BFS- or DFS-like exploration
- **Few knobs** — beyond walk length and window size, no way to adapt the walks to the task

(Other limitations — no node features, transductive only, static graph — are shared with **all** shallow embeddings, and we'll come back to them when we discuss GNNs.)

Section 6 — node2vec

DeepWalk with two new knobs — the algorithm becomes a **family**.

- **Origin** — Grover & Leskovec (KDD 2016); explicitly designed to fix DeepWalk's "one-size-fits-all" walk strategy
- **Core innovation** — **biased** random walks with two interpretable parameters p and q that smoothly interpolate between BFS-like and DFS-like exploration
- **Reference** — node2vec: Scalable Feature Learning for Networks (KDD 2016)

The training pipeline (Skip-gram + negative sampling) is **identical** to DeepWalk. The novelty is entirely in **how walks are generated**.

Two structural properties node2vec can capture

The mapping between walk strategy and what gets preserved is **counter-intuitive** — read carefully (Grover & Leskovec, 2016, §3.1).

1. Structural equivalence (functional roles).

Nodes playing similar *roles* in different parts of the graph should be embedded close together — even if they are far apart in the network.

Example. Two airports, one in Europe and one in Asia, that both serve as international hubs have similar local connection patterns despite no direct link between them.

Captured by **BFS-like, local exploration** — by restricting walks to the immediate neighbourhood, the algorithm samples the local "fingerprint" of each node, and nodes with similar fingerprints end up close in embedding space.

Two structural properties — homophily

Homophily (community structure).

Nodes in the same community should be embedded close together — cliques of friends, dense subgraphs, topical clusters.

Captured by **DFS-like, broader exploration** — long, outward-bound walks reflect a macro-view of the neighbourhood, so nodes that belong to the same dense region tend to co-occur in the same walks.

The single parameter pair (p, q) lets the practitioner choose where on this spectrum to operate.

node2vec — the algorithm

Walk control parameters.

- **Return parameter** p — controls the chance of stepping back to the previous node
  - $p < 1$: walks revisit (stay in tight neighbourhoods)
  - $p > 1$: walks avoid backtracking (explore outward)
- **In-out parameter** q — controls "outward" vs. "inward" exploration
  - $q < 1$: walks venture outward (DFS-like — captures **homophily / communities**)
  - $q > 1$: walks stay near the source (BFS-like — captures **structural equivalence / roles**)

node2vec — pipeline

1. Pick p and q for your task
2. For each node, run r biased random walks of length L
3. Apply Skip-gram with negative sampling to the resulting walk corpus — exactly as DeepWalk does

The biased transition rule

A node2vec walk has **memory of the previous step**. Suppose the walk just moved from t to v and now needs to pick the next node x from v's neighbours. There are exactly three kinds of moves:

Move type	Description	Probability is multiplied by
Return	step back to t	$1/p$
Triangle	jump to a node also adjacent to t	1
Outward	jump to a node *not* adjacent to t	$1/q$

Reading the biased rule

One knob (p) controls *how willing the walk is to backtrack*; the other (q) controls *how willing it is to wander away* from where it came from.

The case $p = q = 1$ recovers DeepWalk's plain uniform walk — DeepWalk is just node2vec with the knobs centered.

(See the original paper, §3.1; the mapping from q to "roles vs. communities" is the opposite of what intuition often suggests.)

Worked example — BFS-like bias

The walk just stepped $t \to v$. Node v has three neighbours:

- $x_1 = t$ — going back
- x_2 — a **triangle neighbour** (also adjacent to t)
- x_3 — an **outward node** (not adjacent to t)

Unnormalised weights: $w(x_1) = 1/p, \;\; w(x_2) = 1, \;\; w(x_3) = 1/q.$

Worked example — BFS-like probabilities
With $p = 0.25,\ q = 4$ the walk stays close to home:

Next node	Weight	Probability
x_1 (back to t)	$1/0.25 = 4$	0.76
x_2 (triangle)	1	0.19
x_3 (outward)	$1/4 = 0.25$	0.05

→ samples each node's **local fingerprint** thoroughly → surfaces **structural equivalence** (e.g. two hub airports with similar local patterns).

Worked example — DFS-like bias

Same setup as the previous slide. Now flip the parameters: $p = 4,\ q = 0.25$. The walk wanders outward.

Next node	Weight	Probability
x_1 (back to t)	0.25	0.05
x_2 (triangle)	1	0.19
x_3 (outward)	4	0.76

→ long trajectories cross dense regions → surfaces **communities (homophily)**.

Reading the two panels together. p controls how willing the walk is to backtrack; q controls how willing it is to leave the local neighbourhood. The two knobs let one algorithm recover **roles** *or* **communities**.

Choosing p, q in practice

Settings from the original paper's Les Misérables case study (Grover & Leskovec, §4.1):

- **Community detection (homophily)** → $p = 1,\ q = 0.5$
- **Role discovery (structural equivalence)** → $p = 1,\ q = 2$
- **Link prediction** → task-dependent; use cross-validated grid search

DeepWalk vs. node2vec — at a glance

Aspect	DeepWalk	node2vec
Walk strategy	uniform random	biased by two parameters p, q
Memory of past	none — Markovian	remembers the previous step
Tunability	none beyond walk length	p, q interpolate BFS ↔️ DFS
Captures	a mix of community + roles	community **or** roles, depending on p, q
Training engine	Skip-gram + negative sampling	*identical*
When to use	quick, robust default	task-specific tuning matters

The big idea. node2vec did not change *how training works* — it changed *what walks see*. Same engine, different data, different structure surfaced.

> **One thing to remember:** p, q interpolate between BFS (roles) and DFS (communities).

UNIVERSITÀ DI TORINO

Section 7 — From nodes to whole graphs

In this section:

- Some tasks need one vector for an **entire graph** — molecule classification, scene-graph recognition, ...
- How do we go from per-node vectors to a single graph-level vector?

By the end: two simple recipes (pooling, supernode) and a pointer to richer methods.

Use case — molecule classification

- Each molecule is a small graph — atoms are nodes, bonds are edges
- We want to predict properties of the molecule as a whole: toxicity, solubility, reactivity, drug-likeness
- The challenge: **molecules vary in size** — we need a *fixed-length* vector (always the same number of entries) regardless of how many atoms the molecule has

This is the prototypical **graph-level** prediction problem, and the same techniques apply to social subgraph classification, scene-graph recognition, code-property graphs, and many other domains.

Method 1 — aggregate node embeddings

The simplest approach: compute the node embeddings, then combine them into one vector for the whole graph. Three common combinators:

- **Sum** — add all the node vectors together
- **Mean** — average them (sum, divided by the number of nodes)
- **Max** — coordinate by coordinate, keep the largest value across all nodes

Each has trade-offs: **sum** depends on graph size; **mean** is size-invariant; **max** captures the most "extreme" features. The choice matters in practice.

Method 2 — virtual supernode

A different trick: change the graph itself.

1. Add a dummy "supernode" s connected to **every** node in the graph
2. Run any node-embedding algorithm on the augmented graph
3. Use $\mathbf{z}_s$ as the graph-level representation

The supernode "sees" every other node during training, so its embedding **aggregates global information automatically** — no manual pooling needed.

Advanced graph-level methods *(out of scope, for reference)*

Beyond simple aggregation, three families capture richer graph-level structure:

- **Hierarchical pooling** *(DiffPool, SAGPool)* — progressively coarsen the graph, grouping similar nodes into super-nodes and re-running the embedding. Multi-resolution representation.
- **Graph kernels** *(Weisfeiler–Lehman)* — compare substructures (graphlets, random walks, subtrees) directly; an embedding is implicitly defined via the kernel matrix.
- **Self-supervised methods** — train to predict graph-level properties (size, diameter, motifs) or reconstruct masked subgraphs; rich representations without task labels.

These methods typically sit on top of GNNs, so they belong to the next lecture.

Section 8 — Evaluating embeddings

In this section:

- Embeddings are an **intermediate** object. How do we judge whether they are good?
- Two canonical protocols: **node classification** and **link prediction**
- How do we look at them — visualisation with t-SNE / UMAP?

By the end: a recipe to take any embedding and benchmark it.

How do we know an embedding is "good"?
Embeddings are intermediate objects — their quality is judged by **downstream task performance**. The two canonical evaluation protocols:

Node classification (transductive).

1. Train embeddings on the full graph (labels not used)
2. Hold out a fraction of node labels
3. Train a simple classifier (logistic regression on $\mathbf{z}_v$) on the visible labels
4. Measure accuracy / F1 on the held-out nodes

Evaluation — link prediction & clustering

Link prediction.

1. Hide a fraction of edges
2. Train embeddings on the reduced graph
3. For each (held-out edge, random non-edge) pair, score with $\mathbf{z}_u^\top \mathbf{z}_v$ (or a more elaborate combiner)
4. Measure ROC-AUC

A third common check is **clustering quality** — apply k-means or HDBSCAN to the embeddings and compare against ground-truth communities (NMI, ARI).

Visualising embeddings

A 64- or 128-dimensional embedding cannot be plotted directly. Two standard tools:

- **t-SNE** (van der Maaten & Hinton, 2008) — preserves local neighbourhoods; good for visualising clusters
- **UMAP** (McInnes et al., 2018) — preserves local *and* some global structure; faster than t-SNE on large graphs

Both compress the high-dimensional embeddings down to **two coordinates per node** so we can plot them. A typical sanity check after training: project the node embeddings and colour them by community / class label — well-trained embeddings produce visibly separated clusters.

Caveat. Distances in t-SNE / UMAP plots are not metrically faithful — use them for qualitative inspection, not quantitative claims.

UNIVERSITÀ DI TORINO

Section 9 — Limitations and the road to GNNs

In this section:

- Where do shallow embeddings break down?
- What do **Graph Neural Networks** add that shallow methods can't?

By the end: a clean motivation for the next lecture.

Limitations of shallow embeddings

1. Transductive only. An embedding is a learned parameter — there is no rule that produces one for a new node. Adding a node, edge, or feature requires retraining. A blocker for any evolving system: recommenders, social networks, fraud detection.

2. Limited structural expressiveness. Random walks miss certain higher-order patterns (e.g. counting triangles); two non-isomorphic graphs can produce the same walk distribution and hence the same embeddings.

3. Node and edge features are ignored. Real networks come with rich metadata — user profiles, post text, atom types, edge timestamps — and shallow methods discard all of it. For feature-rich domains (molecules, knowledge graphs, content recommendation), this is a heavy cost.

Graph Neural Networks — beyond shallow embeddings

The next lecture introduces GNNs, which address each of the limitations above:

- **Richer information processing** — leverage *both* graph structure *and* node/edge features
  In a social network: combine friendship links *with* user profiles and post content
  Critical for molecules, knowledge graphs, and any feature-rich domain
- **Adaptable to dynamic environments** — handle evolving graphs and unseen nodes
  No retraining when the graph changes; the encoder is a function, not a table
  Essential for real-time systems (recommenders, fraud detection, traffic forecasting)

GNNs — beyond shallow embeddings (cont.)

- **Better on complex patterns** — message-passing layers capture higher-order dependencies Multiple layers learn hierarchical patterns, much as ConvNets do for images
- **Production-scale** — GNNs at Pinterest, Twitter, Google, Alibaba run on graphs with billions of nodes

Summary — the journey, told as a story

We started in a familiar place: doing ML on networks meant **hand-crafting features** — degree, PageRank, clustering — and feeding them to a classifier. That worked, but the features had a ceiling and didn't transfer.

We asked: **could the features be learned instead?** The answer became a unifying scaffold — the **encoder–decoder framework**. Every method is a choice of: how to turn nodes into vectors, what similarity to preserve, how to decode that similarity, and how to measure mismatch.

To make the scaffold scale, we borrowed from language: **random walks** play the role of sentences, **Skip-gram** turns walks into a learning target, and **negative sampling** keeps training cheap.

Summary — the journey (continued)
That recipe gave us **DeepWalk** — the first complete algorithm — and then **node2vec**, which kept the same engine but added two knobs p, q to bias the walks between BFS (roles) and DFS (communities).

We saw how to scale up to whole graphs (pooling, supernodes), how to evaluate embeddings (link prediction, node classification), and where these methods break: they are **transductive** and **feature-blind**.

That gap is what motivates the next lecture — **Graph Neural Networks** — which keep the encoder–decoder spirit but make the encoder a function over node features, lifting both limitations.

> **One thing to remember from the whole lecture:** every method = encoder + similarity + decoder + loss; everything else is a choice.

Q&A